

Московский государственный технический университет
им. Н.Э. Баумана

Утверждаю:
Большаков С.А.
« » _____ 2026 г.

Курсовая работа по курсу «Системное программирование»
«Резидентная программа (TSR)»

Техническое описание

(вид документа)

писчая бумага

(вид носителя)

9

(количество листов)

ИСПОЛНИТЕЛИ:
студент группы ИУ5-42Б

Воробьев Е. А., « » _____ 2026 г.

Москва — 2026

СОДЕРЖАНИЕ

1. Общие сведения о программном обеспечении.....	3
2. Модульная структура программного обеспечения.....	3
3. Описание модулей программного обеспечения.....	3
4. Данные и файлы данных программного обеспечения.....	3
5. Основные алгоритмы программного обеспечения.....	4
6. Процедуры и функции программного обеспечения.....	6
7. Векторы прерываний программного обеспечения, переопределяемые в программе.....	7

1. ОБЩИЕ СВЕДЕНИЯ О ПРОГРАММНОМ ОБЕСПЕЧЕНИИ

Исходный код, язык: Assembler.

Компилятор: Turbo Assembler Version 3.1.

Сборщик: Turbo Link Version 5.1.

Отладчик: Turbo Debugger Version 3.1.

Исполняемый код: файл `tsr.com` (2127 байт).

Исходный код: файл `tsr.asm` (22 470 байт).

2. МОДУЛЬНАЯ СТРУКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Программа делится на резидентную и нерезидентную (инициализирующую) части. Резидентная часть реализует функционал данного программного обеспечения. Нерезидентная часть нужна для инициализации резидентной части, обработки параметров командной строки и выгрузки резидента по повторному запуску программы.

См. документ «Модульная структура программы».

3. ОПИСАНИЕ МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

См. документ «Схема взаимодействия с аппаратурой».

4. ДАННЫЕ И ФАЙЛЫ ДАННЫХ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Программа не использует внешних файлов данных. Все данные размещены в сегменте кода в начале программы после `org 100h`.

Модуль `tsr.asm`:

Имя	Размер	Хранящиеся данные
<code>ignoredChars</code>	66 байт	список русских букв (cp866) для блокировки при F2
<code>ignoredLength</code>	<code>equ</code>	длина строки <code>ignoredChars</code>
<code>ignoreEna</code>	1 байт	флаг функции игнорирования ввода (F2)

Имя	Размер	Хранящиеся данные
bled		
translateFrom	5 байт	латинские клавиши T, ;, P, B, R
translateTo	5 байт	русские буквы ЕЖЗИК
translateLength	equ	длина строки translateTo
translateEnabled	1 байт	флаг функции русификации (F1)
signaturePrintingEnabled	1 байт	флаг функции вывода подписи (F8)
cursiveEnabled	1 байт	флаг режима курсива буквы «З» (F9)
cursiveSymbol	16 байт	битовая матрица 8×16 курсивной буквы «З»
charToCursiveIndex	1 байт	код символа ср866 для замены — «З» (97h)
savedSymbol	16 байт	сохранённое начертание буквы «З»
old_int9hOffset	2 байта	адрес (смещение) старого обработчика INT 09h
old_int9hSegment	2 байта	адрес (сегмент) старого обработчика INT 09h
old_int1ChOffset	2 байта	адрес (смещение) старого обработчика INT 1Ch
old_int1ChSegment	2 байта	адрес (сегмент) старого обработчика INT 1Ch
old_int2FhOffset	2 байта	адрес (смещение) старого обработчика INT 2Fh
old_int2FhSegment	2 байта	адрес (сегмент) старого обработчика INT 2Fh
unloadTSR	1 байт	1 — выгрузить резидент
notLoadTSR	1 байт	1 — не загружать резидент
counter	2 байта	счётчик тиков таймера INT 1Ch
printDelay	equ	задержка перед выводом подписи (5 секунд)
printPos	2 байта	положение подписи: 0 — верх, 1 — центр, 2 — низ
signatureLine1	58 байт	первая строка подписи — « Воробьев Е. А. ... »
signatureL	58 байт	вторая строка подписи — « ИУ5-42Б ...

Имя	Размер	Хранящиеся данные
ine2		»
signatureLine3	58 байт	третья строка подписи — « Вариант № 5 ... »
helpMsg	~400 байт	текст справки по ключу /?
tableTop	58 байт	верхняя рамка таблицы подписи
tableBottom	58 байт	нижняя рамка таблицы подписи
installedMsg	19 байт	«Резидент загружен!»
alreadyInstalledMsg	22 байт	«Резидент уже загружен»
noMemMsg	20 байт	«Недостаточно памяти»
removedMsg	16 байт	«Резидент выгружен»
noRemoveMsg	29 байт	«Не удалось выгрузить резидент»
errorParamMsg	36 байт	«Ошибка параметров командной строки»
f1_txt..f4_txt	по 2 байта	подписи кнопок F8, F9, F1, F2
fx_length	equ	длина строк f1_txt..f4_txt

5. ОСНОВНЫЕ АЛГОРИТМЫ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Ход инициализации. Устанавливается требуемый видеорежим для вывода текстовых сообщений во время работы резидента, вызывается обработчик параметров командной строки, затем, если программа запущена без параметров и резидент отсутствует в памяти, происходит установка резидента и удаление из ОП кода ниже метки `_initTSR`. Если задан флаг `/?`, выводится справка по работе с программой. Если же программа запущена без параметров и резидент уже находится в памяти, то выполняется выгрузка резидента через `INT 2Fh` с восстановлением старых обработчиков прерываний (требование группы ИУ5-42Б).

Обработчик `new_int2Fh`. Вначале проверяется, равен ли регистр `АН` значению `0FFh`. Если это так — это наша подфункция, и при `AL=0` в `АН` заносится `'i'` как признак того, что резидент уже загружен в память. При

AL=1 выполняется выгрузка резидента из памяти — восстанавливаются старые обработчики прерываний INT 09h, INT 1Ch, INT 2Fh, освобождается PSP-окружение и тело резидента.

Обработчик new_int9h. Из порта 60h считывается скан-код нажатой клавиши. При нажатии функциональных клавиш F8, F9, F1, F2 меняют свои значения флаги соответствующих режимов (вывод подписи, курсив буквы «З», русификация клавиатуры, ограничение ввода русских букв), а также обновляется их индикация в правом нижнем углу экрана. Затем вызывается стандартный обработчик данного прерывания. Если введённый символ входит в множество игнорируемых (русские буквы при F2 активном), он удаляется из буфера клавиатуры. Если входит в множество translateFrom — заменяется на символ с тем же индексом из множества translateTo (при F1 активном).

Обработчик new_int1Ch. В самом начале работы обработчика производится вызов старого обработчика прерывания INT 1Ch. В случае, если флаг signaturePrintingEnabled установлен в true, производится сравнение счётчика counter вызовов прерывания системой с числом $\text{printDelay} * 1000 / 55 + 1$, где printDelay — задержка в секундах (5). Если эти числа равны, далее производится печать информации об исполнителе курсовой работы на экран; иначе counter увеличивается на 1.

Функция вывода подписи (printSignature). Читается текущее положение курсора на экране и запоминается в стеке. Далее происходит выбор положения подписи на экране (верх, центр или низ) на основе значения переменной printPos. Для варианта № 5 printPos равно 0 — подпись выводится в верхнюю часть экрана. В каждом случае устанавливаются значения регистров DH и DL, хранящие информацию о строке и столбце соответственно. Затем эта информация используется для вывода построчно подписи, содержащей верхнюю рамку, три строки информации об исполнителе и нижнюю рамку. Восстанавливается положение курсора из

стека. Вызывается функция `changeFx` для прекращения индикации вывода подписи.

Функция вывода индикации (`changeFx`). Читается текущее положение курсора и запоминается в стеке. Далее происходит перебор всех четырёх флагов состояний — `signaturePrintingEnabled`, `cursiveEnabled`, `translateEnabled`, `ignoreEnabled`. В случае, если флаг установлен в `true`, производится печать наименования функциональной клавиши соответствующего флага в правом нижнем углу экрана на зелёном фоне, иначе — на красном. Восстанавливается положение курсора из стека.

Функция проверки командной строки (`commandParamsParser`). В регистр `SI` помещается смещение `80h` (адрес параметров командной строки в `PSP`). Читается количество символов в параметрах. Если их количество равно 0 — выходим. Иначе идёт цикл до тех пор, пока не будет прочитан символ возврата каретки. Если одним из параметров является `/?`, выводится справка по использованию программы и устанавливается флаг того, что загружать резидент не надо (`notLoadTSR`).

Функция, меняющая изображение символа с курсива на обычное и наоборот (`setCursive`). В стек сохраняются регистры `AX` и `ES`, в регистр `ES` загружается значение регистра `CS`. Далее, если флаг `cursiveEnabled` установлен в `true`, происходит сохранение текущего изображения изменяемого символа и последующая замена на новое. Номер изменяемого символа содержит переменная `charToCursiveIndex` (для варианта № 5 — `0x97` «З»). С помощью процедуры `saveFont` определяется адрес текущей таблицы символов. Затем, добавляя необходимое значение к регистру `BP` (`16 * charToCursiveIndex`), находится адрес нужного символа и сохраняется 16 байт таблицы его изображения в переменную `savedSymbol`. После чего в регистр `CX` заносится 1, в `DL` устанавливается номер изменяемого символа, в `BP` перемещается адрес таблицы нового символа из переменной `cursiveSymbol`. Происходит вызов функции `changeFont`. Если флаг `cursiveEnabled` не установлен — происходит восстановление старого изображения символа.

См. документ «Блок-схема алгоритма программы».

6. ПРОЦЕДУРЫ И ФУНКЦИИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Название	Входные данные	Выходные данные	Описание
new_int9h	—	—	Обработчик прерывания 09h
new_int1Ch	—	—	Обработчик прерывания 1Ch
new_int2Fh	АН=0FFh, AL=0 — проверка; AL=1 — выгрузка	АН='i', если резидент в памяти	Обработчик прерывания 2Fh
printSignature	—	—	Процедура вывода подписи (ФИО, группа, вариант)
setCursive	—	—	Меняет начертание символа «З» с курсива на обычное и наоборот в зависимости от флага cursiveEnabled
changeFont	DL = номер символа для замены; CX = количество заменяемых символов; ES:BP = адрес таблицы	—	Функция смены начертания символа в шрифте VGA через INT 10h функция 11h
saveFont	ВН — тип возвращаемой символьной таблицы	ES:BP — адрес таблицы; CX — байт на символ; DL — количество экранных строк	Функция чтения текущей таблицы шрифта

Название	Входные данные	Выходные данные	Описание
			VGA через INT 10h функция 30h
commandParamsParser	—	unloadTSR, notLoadTSR	Процедура проверки параметров командной строки
changeFx	—	—	Процедура вывода состояния клавиш F8, F9, F1, F2 в правом нижнем углу экрана

7. ВЕКТОРЫ ПЕРЕРЫВАНИЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ, ПЕРЕОПРЕДЕЛЯЕМЫЕ В ПРОГРАММЕ

В программе переопределяются три вектора прерываний:

1. **INT 09h** — обработка нажатия клавиш. Источник — аппаратное прерывание IRQ1 от клавиатурного контроллера 8042.
2. **INT 2Fh** — мультиплексор DOS. Используется для проверки наличия программы в памяти, а также для выгрузки резидентной части программы.
3. **INT 1Ch** — пользовательский таймерный обработчик. Используется для подсчёта времени, прошедшего после нажатия функциональной клавиши F8, для последующего вывода сообщения-подписи.

Сохранение старых обработчиков выполняется через INT 21h функция 35xx, установка новых — через 25xx. При выгрузке (по повторному запуску `tsr.com` без параметров) все три адреса восстанавливаются в исходное состояние.